

Weekly Plan – 2026-07-27 to 2026-08-01

Production vs Staging – Gap Analysis

UPDATE 2026-07-28: All 6 sections now on upstream/staging at commit 964152e3. Section 4 (CHANNEL_INVALID auto-disable) DONE via Emma’s PR #49. Section 5 (Realtime reconnect) DONE via our commit. Martins’ fixes (8 commits: basket management, trade reconciliation, ensureSignalRow, order close audit, SL/TP validation, dashboard i18n) backported from prod. Staging now contains everything dev has — ready for comprehensive testing. See test checklist below.

Last production merge: PR #52 (13877d82, staging → main) on Jul 28. Staging now contains all fixes (Sections 2-6) plus Martins’ 8 commits backported from production. dev and staging are identical at 964152e3.

Staging test checklist at: docs/staging-test-checklist.md

Commits	What	Impact on Production
b3a8f38a + 0218a215 + f1981ad5	TIMEOUT death spiral fix	Critical — stops zombie listeners
ef01e883	QR login AUTH_KEY_UNREGISTERED loop fix	Critical — stops infinite QR retry
6d3e065e + 17188c3b	Auth death spiral + session persistence	Already on both branches (different hashes)
PR #47 (3 commits)	AUTH_KEY_DUPLICATED at scale	Critical — graceful shutdown, reconnect delays, max retry limit
PR #48 (1 commit) + patch script security	BinaryReader crash guard + version assertion	Critical — prevents silent connection corruption
Our commit ff483ae7	Remaining items 2.4 (connect-trace) + 3.2 (raw error fallback)	Defense in depth
Our commit 5b92344a	Dockerfile fix (copy scripts/ before npm ci)	Required — without it, postinstall fails in Docker
PR #44 + PR #45 (4 commits)	Pipeline observability + latency	Performance only

Production log errors mapped to root cause and fix status:

Production error	Root cause	Fixed on staging?	Fixed on production?
TIMEOUT death spiral	autoReconnect:false + no handler	YES (b3a8f38a)	NO
QR login AUTH_KEY_UNREGISTERED loop	onError returns false	YES (ef01e883)	NO
BinaryReader crash (readUInt32LE)	No guard for undefined body	YES (PR #48 + raw fallback)	NO
AUTH_KEY_DUPLICATED at scale	TCP race on deploy/restart	YES (PR #47)	PARTIAL (0daf7cb2)
CHANNEL_INVALID silent loop	No auto-deactivation	NO	NO
Stale channel usernames	No cleanup	NO	NO
Realtime subscription drop	No reconnect after CHANNEL_ERROR	NO	NO

Section 1 – Merge staging into production

Problem: Staging is 23 commits ahead of production. Production is missing critical fixes for TIMEOUT death spirals, QR login loops, AUTH_KEY_DUPLICATED at scale, and BinaryReader crashes — all causing signal loss or zombie listeners.

Plain English fix: Merge everything from staging into production in one shot.

Technical detail: upstream/staging (5b92344a) is **23 commits** ahead of upstream/main (production).

The critical fixes on staging that production doesn't have: - TIMEOUT death spiral fix (b3a8f38a) - QR login AUTH_KEY_UNREGISTERED fix (ef01e883) - AUTH_KEY_DUPLICATED full fix — PR #47 (3 commits: af12737d, 248844be, 0df31750) - BinaryReader crash guard — PR #48 (0df31750) + raw error string fallback (ff483ae7) - Connect-trace logging + Dockerfile fix (ff483ae7, 5b92344a)

Also includes pipeline observability (PR #45) and range-layer latency fix (PR #44).

The auth session persistence fix and auth death spiral fix are already on both branches via different merge paths.

Files changed: worker/src/userListener.ts (+89/-1), worker/src/telegramClient.ts (+4/-0), worker/scripts/apply-node-module-patches.cjs (+36/-0), worker/Dockerfile (+2/-0), worker/src/authKeyDuplicatedRecovery.ts, worker/src/sessionManager.ts, worker/src/sessionLease.ts, worker/src/workerShutdown.ts, plus PR #44 and #45 changes.

Plain English: Production is missing fixes for 4 different crash/loop bugs. Each one causes signal loss for a different reason. Merging staging brings all fixes at once — including the newly added Dockerfile fix that was required just to get the build passing.

How it affects users: Currently all production users are vulnerable to all 4 bugs. Users hit TIMEOUT → signal loss until restart. Users with QR login → infinite spinner. Users hit by BinaryReader crash → 30s signal gaps. Users hit by AUTH_KEY_DUPLICATED → signal loss for 10-90s per deploy.

Expected outcome: Zero TIMEOUT death spirals. Zero QR login loops. Zero BinaryReader crashes. AUTH_KEY_DUPLICATED recovery in ~30s instead of never.

- ☒ **1.0** Promote dev → staging (DONE — both at 5b92344a)
- ☐ **1.1** Verify pipeline_ts migration exists on production Supabase (staging migration 20260724120000_signals_pipeline_ts.sql)
- ☐ **1.2** Merge upstream/staging into upstream/main (git push upstream staging:main)
- ☐ **1.3** Trigger Railway redeploy from main
- ☐ **1.4** Monitor production logs for 1h – verify TIMEOUT errors stop appearing

Section 2 – Fix AUTH_KEY_DUPLICATED at scale

Problem: Every Railway deploy kills the old worker before its Telegram connections fully close. The new worker connects with the same auth keys, Telegram sees duplicates, and kicks sessions off. Users miss signals for 10-90s per deploy. Some sessions never recover.

Plain English fix: Gracefully close all Telegram connections before the old worker shuts down, so the new worker doesn't create duplicate connections. Wait longer before retrying so Telegram has time to release the old keys.

Technical detail: A SIGTERM handler already exists at worker/src/index.ts:258. It calls shutdown() which runs sessionManager.disconnectAll() (line 252) then waits TELEGRAM_SHUTDOWN_DRAIN_MS (default 8s, line 254). The handler is correct in structure but the drain window is too short for 53 connections.

The OS keeps TCP connections in TIME_WAIT state for 30-60s. The new worker connects before the old connections fully close, Telegram sees duplicate auth keys, and sends AUTH_KEY_DUPLICATED.

Caveat: The drain timeout is capped at 10_000ms via Math.min(10_000, ...). Simply increasing TELEGRAM_SHUTDOWN_DRAIN_MS to 30s is insufficient — the cap must also be removed or increased (item 2.1 must adjust both values).

Current recovery takes 10s + 4 retries (15s, 30s, 30s, 60s deferred) from authKeyDupReconnectDelaysMs() in worker/src/authKeyDuplicatedRecovery.ts. The named constant AUTH_KEY_DUP_RECONNECT_DELAY_MS lives in worker/src/userListener.ts:190 (default 10s), but worker/src/sessionManager.ts:784 passes a hardcoded 8_000 instead of referencing it.

Fix (status as of Emma's PR #47 on upstream/dev):

- **2.1: DONE** — Drain timeout raised to 30s default, Math.min(10_000, ...) cap raised to 120s. Implemented in worker/src/workerShutdown.ts + worker/src/index.ts:252-254.
- **2.2: DONE** — Default raised from 10s to 30s, centralized in authKeyDupReconnectDelayMs() in worker/src/authKeyDuplicatedRecovery.ts:38-42.
- **2.3: DONE** — Hardcoded 8_000 replaced with authKeyDupReconnectDelayMs() at worker/src/sessionManager.ts:805.
- **2.4: DONE** — connectTrace() method added to UserListener (26 calls across start/stop/forceReconnect/getDialogs). telegramClient.ts buildClient() now emits [telegram-conn] event=build_client with redacted session fingerprint.

- **2.5: DONE** — `releaseOwnedSessionLeases()` added to `worker/src/sessionLease.ts:158-172`, called at end of `disconnectAll()` in `sessionManager.ts:1199`.
- **2.6: DONE** — Max retry limit added (`authKeyDupMaxRecoveryAttempts()`, default 10). After exhaustion, `onAuthKeyDuplicatedRecoveryExhausted` calls `sessionManager.invalidateTelegramSession()`. Old deferred retry loop removed.

Plain English: When Railway deploys a new version, it kills the old worker before its Telegram connections are fully closed. Telegram sees two connections using the same auth key and kicks one off. With 53 sessions, most listeners spend 10-90 seconds reconnecting after every deploy. During that window, the user receives no trade signals. If all 4 retries fail, the session is permanently dead and the user has to manually reconnect in the UI.

How it affects users: Every deploy causes 10-90 seconds of missed signals per user. Some users get permanently disconnected. This is the #1 stability problem.

Expected outcome: Old worker gracefully closes all Telegram connections before exiting (tells Telegram “goodbye”). New worker waits long enough for keys to be released. Zero missed signals during deploys.

- ☑ **2.1** Increase drain timeout from 8s to 30s in existing SIGTERM handler (`index.ts:254`) — DONE (PR #47)
 - ☑ **2.2** Increase `AUTH_KEY_DUP_RECONNECT_DELAY_MS` from 10s to 30s (`userListener.ts:191`) — DONE (PR #47)
 - ☑ **2.3** Fix hardcoded `8_000` to match delay in `sessionManager.ts` (`sessionManager.ts:784`) — DONE (PR #47)
 - ☑ **2.4** Add connect-trace logging in `telegramClient.ts` and `userListener.ts` — DONE (`telegramClient.ts buildClient()` added)
 - ☑ **2.5** Add `releaseAllLeases()` to `sessionManager`, call after `disconnectAll()` in shutdown — DONE (PR #47, named `releaseOwnedSessionLeases`)
 - ☑ **2.6** Add max retry limit to `AUTH_KEY_DUPLICATED` reconnect loop — invalidate session after 10 cycles (~10 min) — DONE (PR #47)
-

Section 3 – Fix BinaryReader crash

Problem: Telegram sometimes returns messages with empty body data. The code tries to read it without checking if it exists and crashes. The crash is caught silently but the connection is now corrupted, causing 30+ second gaps in signal delivery. With 53 sessions this hits every few hours.

Plain English fix: Before trying to read incoming Telegram data, check that the data actually exists. If it’s empty, skip it instead of crashing. If a crash somehow happens anyway, force a full reconnect so the connection isn’t silently corrupted.

Technical detail: `MTPProtoSender._handleRPCResult()` (`worker/node_modules/telegram/network/MTPProtoSender.js:545`) creates new `BinaryReader(result.body)` without checking `result.body` when no matching pending RPC state exists (the `!state` branch). Line 568 (the `state` branch) has the same guard gap.

When `MTPProto` returns garbled data, `RPCResult.fromReader` returns `{ body: undefined, error: undefined }`. The `BinaryReader` constructor sets `this.stream = undefined`, then `readUInt32LE()` on `undefined` crashes with “Cannot read properties of undefined”.

The `_recvLoop` inner catch (line 441, log at line 451) logs “Unhandled error while receiving data” (non-`RPCError` only; `RPCError` is handled without logging) and the loop continues — but the connection is now corrupted. A separate outer catch (line 375) logs the same message but returns instead of continuing, though neither path repairs the connection.

Fix (status as of Emma’s PR #48 on upstream/dev):

- **3.1: DONE** — Guard added at both `BinaryReader` sites via `worker/patches/telegram+2.26.22.patch`. Applied by `worker/scripts/apply-node-module-patches.cjs` postinstall hook. Guard validates `Buffer.isBuffer(body) && body.length > 0` (stronger than just `if (!result.body)`). 5 test cases in `gramjsMalformedRpcResultPatch.test.ts`.
- **3.2: DONE** — `client.onError` handler now catches both `GRAMJS_MALFORMED_RPC_RESULT` (custom error from 3.1) AND raw `readUInt32LE / Cannot read properties of undefined` error message strings as fallback in case the patch doesn’t trigger. Both paths route to `noteMalformedRpcResult` which manages rate-limited reconnect.

Plain English: Sometimes Telegram sends back data with an empty body. The code tries to read it without checking if it exists and crashes. The crash is caught silently but the connection is now corrupted – messages may be lost for 30+ seconds until the watchdog recovers. With 53 sessions, this happens every few hours. Staging doesn’t see it because it’s probabilistic and only 3 sessions.

How it affects users: Trade signals stop arriving for 30+ seconds with no error shown. The user doesn’t know anything is wrong.

Expected outcome: Empty data is discarded cleanly. Zero crashes from malformed responses. Zero silent connection

corruption.

- ☒ **3.1** Add guard clause before BinaryReader creation in MTProtoSender (both line 545 and 568) — DONE (PR #48, via patch)
 - ☒ **3.2** Register BinaryReader-style errors with watchdog reconnect — DONE (raw error strings readUInt32LE and Cannot read properties of undefined added as fallback in onError handler)
-

Section 4 – Fix CHANNEL_INVALID silent loop

Problem: When a Telegram channel is deleted or the bot is removed, every read attempt returns CHANNEL_INVALID. The worker retries every 3-10 seconds forever, wasting API quota, burning flood wait limits, and never telling the user their channel is dead.

Plain English fix: Track how many times each channel fails with CHANNEL_INVALID. After 5 failures, automatically disable the channel in the database so the worker stops retrying it forever and the user sees it's broken in the UI.

Technical detail: resolveChannelPeer() (worker/src/userListener.ts:3175) catches all errors but has no CHANNEL_INVALID-specific handling.

ensureJoinedPublicChannel() (line 2896) suppresses USER_ALREADY_PARTICIPANT, CHANNELS_TOO_MUCH, INVITE_HASH_EMPTY but not CHANNEL_INVALID.

Result: deleted/renamed channels retry every safety poll (10s) and fast poll (3s) forever. No in-memory counter, no DB deactivation.

Fix:

- **4.1:** Add Map<channelId, { consecutiveFailures, lastFailureAt }> in userListener.ts
- **4.2:** Increment on CHANNEL_INVALID in resolveChannelPeer() and pollChannelNewMessages()
- **4.3:** After 5 consecutive failures: update telegram_channels set is_active = false, log, remove from monitored set
- **4.4:** Same treatment for “No user has ... as username” errors from ensureJoinedPublicChannel()

Plain English: When a Telegram channel is deleted or the bot is removed, every attempt to read it returns CHANNEL_INVALID. The worker retries every 3-10 seconds forever – wasting API quota, generating log noise, and never telling the user. With 53 users x 2-5 channels each, dead channels burn significant resources and cause flood wait limits to kick in faster for everyone.

How it affects users: The user sees “channel is configured” in the UI but receives no signals from it. No error, no indication. Meanwhile, retrying the dead channel slows down processing for working channels.

Expected outcome: After 5 consecutive failures, the channel is auto-disabled. The UI shows “channel disconnected” so the user can fix it. Worker stops wasting resources.

- ☒ **4.1** Track consecutive CHANNEL_INVALID per channel in memory — DONE (PR #49, ChannelInvalidFailureState Map in userListener.ts:403)
 - ☒ **4.2** After 5 failures, auto-deactivate in DB — DONE (PR #49, disableInvalidChannel at userListener.ts:575, DB update + remove from monitoring + auto_disabled event)
 - ☒ **4.3** Same for stale usernames — DONE (PR #49, isConfirmedChannelInvalidError includes USERNAME_NOT_OCCUPIED/USERNAME_INVALID, handled in ensureJoinedPublicChannel)
-

Section 5 – Fix Realtime subscription reconnect gap

Problem: The Supabase Realtime WebSocket drops every 20-40 minutes. When it drops, the code never reconnects because of a guard that says “don’t subscribe twice.” Channel updates and auth events go undelivered for up to 30 seconds.

Plain English fix: When the Supabase WebSocket connection drops (which it does every 20-40 minutes), automatically re-subscribe instead of staying disconnected forever.

Technical detail: subscribeToChannelChanges() (sessionManager.ts:335) guards with if (this.channelChannel) return (line 336), and subscribeToAuthPendingChanges() (line 363) guards with if (this.authPendingChannel) return (line 364).

Supabase Realtime’s .subscribe() callback fires on status changes, but CLOSED and CHANNEL_ERROR only log warnings without clearing the reference. The reference is never nulled, so the subscription is never recreated.

Staging logs show CHANNEL_ERROR every 20-40 min – the subscription dies silently each time.

Fix:

- **5.1:** In the subscribe callback on CLOSED/CHANNEL_ERROR: set `this.channelChannel = null`, then `setTimeout(() => this.subscribeToChannelChanges(), 5000)`
- **5.2:** Add 60s health-check timer calling `supabase.getChannels()` – if subscriptions missing, recreate

Plain English: The worker listens for channel changes and auth requests via a Supabase WebSocket. This WebSocket drops every 20-40 minutes (normal behavior). When it drops, the worker never reconnects because of a guard that says “don’t subscribe twice.” Channel updates and auth events go unnoticed for up to 30 seconds (until the next periodic check).

How it affects users: When a user adds a channel in the web app, there’s a 0-30 second delay before the worker picks it up. Usually fine, but occasionally the update can be missed entirely.

Expected outcome: Subscriptions auto-recover within 5 seconds. Channel changes and auth events propagate instantly.

- ☒ **5.1** On CLOSED/CHANNEL_ERROR callback, clear guard and retry after 5s — DONE (sessionManager.ts:365-368 and 394-397)
- ☒ **5.2** Add 60s health-check timer for subscriptions — DONE (sessionManager.ts:402-421, started at line 261, stopped in stopChannelListenerServices/disconnectAll)

Section 6 – Validate at scale before production rollout

Problem: Staging runs 3 sessions. Production runs 53. Bugs like BinaryReader crashes and AUTH_KEY_DUPLICATED races only appear under load. Rolling out untested fixes to 53 users risks worse downtime than the current state.

Plain English fix: Copy production channel configs and create 50 synthetic session records on staging (with fake session strings). This exercises the session manager, channel polling, and signal pipeline at production-like scale without needing real Telegram connections for the synthetic users. The 3 real staging Telegram sessions continue working normally for connectivity validation.

Technical detail: Staging runs 3 sessions on 1 shard/1 replica. Production runs 53 sessions. BinaryReader crashes, AUTH_KEY_DUPLICATED races, and listener start races are probability-dependent. 3 sessions may not trigger them in a day, while 53 sessions hit them hourly.

We cannot copy real Telegram session strings from production to staging – doing so would cause AUTH_KEY_DUPLICATED on production. Instead, we create synthetic sessions with blank/random session strings. The session manager will try to connect them, fail gracefully (invalid session), and log the failure. This still exercises the full session lifecycle: startup, lease acquisition, connect attempt, error handling, retry logic, and lease renewal.

For the 3 real staging Telegram sessions, we keep them connected and monitor AUTH_KEY_DUPLICATED counts, TIMEOUT rates, and signal processing latency.

What data to use and where to get it:

Data	Source	How to get it	What to do with it
Channel configs	Production telegram_channels	SELECT * FROM telegram_channels WHERE is_active = true via Supabase dashboard or psql	Insert into staging telegram_channels, re-map user_ids to staging user IDs
User profiles	Production user_profiles	SELECT id, copier_paused, ... FROM user_profiles	Create corresponding rows in staging, blank out any PII
Session records	Production telegram_sessions	SELECT user_id, is_active FROM telegram_sessions WHERE is_active = true (exclude session_string)	Insert into staging telegram_sessions with session_string = '' (will fail to connect, which is fine)
Signal volume	N/A	Not needed for this test – the signal pipeline is exercised by the real channel polling	

How to set it up:

1. Export channel configs and user profiles from production Supabase to CSV
2. Insert into staging Supabase with new staging user IDs (map production user_id -> staging user_id)
3. Create 50 session rows in staging telegram_sessions with session_string = '' and is_active = true
4. Restart the staging Railway listener – it will pick up all 53 sessions, connect 3 real ones, and fail 50 gracefully
5. Monitor for 4 hours

What this validates: - Session manager starts 53 listeners without crashing - Lease system handles 53 concurrent leases - Channel polling loop runs at scale (many channels per session manager) - Memory and CPU stay stable under 53-session load - No regressions on the 3 real Telegram connections (AUTH_KEY_DUPLICATED count, TIMEOUT rate, signal latency) - Entity cache warmup at scale - Realtime subscription handling with many channel changes

What this does NOT validate (and why that's okay): - Actual AUTH_KEY_DUPLICATED recovery – validated by Section 2 fixes (SIGTERM drain) which we verify by monitoring the first production deploy: if the drain timeout is long enough, AUTH_KEY_DUPLICATED errors should drop significantly. No canary needed – rollback is a single git revert + redeploy. - BinaryReader crash frequency – validated by Section 3 fix (guard clause) which is a static code change, not load-dependent - Flood wait limits – only triggered by real Telegram activity, not reproducible in staging

Fix:

- **6.1:** Export production channel configs and user profiles (excluding session strings)
- **6.2:** Insert into staging Supabase with 50 blank session records
- **6.3:** Restart staging listener, monitor for 4 hours
- **6.4:** If staging passes, full production rollout (merge staging -> main, deploy, monitor for 2h)

Expected outcome: Confidence each fix works at production scale before touching real users.

- ☒ **6.1** Export production channel configs and user profiles (exclude session strings) — DONE
- ☒ **6.2** Insert into staging with 50 blank session records — DONE (59 synthetic sessions, 170 channels remapped, all PII blanked)
- ☐ **6.3** Monitor staging listener for 4 hours — PENDING (see docs/staging-test-checklist.md #6)
- ☐ **6.4** Full production rollout — PENDING (blocked on 6.3)